Array Subscripts & Processing

Advanced Array Operations in C Programming

Subscript Functions Processing

🔢 Array Subscripts

🔢 To access **individual elements in an array**, a subscript is used.

Example:

```
scanf("%d", &stud_marks[i]);
```

- **Subscript Definition**
  Integer type constant or variable name whose value ranges from 0 to SIZE-1

- **String Example**
  char country[] = "India"; // size 5 due to null character '\0'

📈 Finding Maximum Marks

📈 Program to find the maximum marks among the marks of 10 students.

```
#include
#define SIZE 10 /* SIZE is a symbolic constant */
main() {
   int i = 0;
   int max = 0;
   int stud_marks[SIZE]; /* array declaration */

   /* enter the values of elements */
   for(i = 0; i < SIZE; i++) {
      printf("Student no. =%d", i+1);
      printf("Enter marks out of 50:");
      scanf("%d", &stud_marks[i]);
   }

   /* find maximum */
   for(i = 0; i < SIZE; i++) {
      if(stud_marks[i] > max)
         max = stud_marks[i];
   }

   printf("\n\nThe maximum of the marks obtained among all 10 students is: %d", max);
}
```

Output:

```
Student no. = 1 Enter marks out of 50: 10
Student no. = 2 Enter marks out of 50: 17
Student no. = 3 Enter marks out of 50: 23
Student no. = 4 Enter marks out of 50: 40
Student no. = 5 Enter marks out of 50: 49
Student no. = 6 Enter marks out of 50: 34
Student no. = 7 Enter marks out of 50: 37
Student no. = 8 Enter marks out of 50: 16
Student no. = 9 Enter marks out of 50: 08
Student no. = 10 Enter marks out of 50: 37

The maximum of the marks obtained among all 10 students is: 49
```

🔄 Passing Arrays to Functions

🔄 To pass a **single-dimension array as an argument** in a function, you would have to declare a formal parameter in one of three ways.

- **Way 1: Pointer Parameter**

```
void myFunction(int *param) {
    // ...
}
```

- **Way 2: Sized Array Parameter**

```
void myFunction(int param[10]) {
    // ...
}
```

🔹 **Way 3: Unsized Array Parameter**

```
void myFunction(int param[]) {
    // ...
}
```

💡 **Key Point:** All three declaration methods produce similar results because each tells the compiler that an integer pointer is going to be received.

📊 getAverage Function Example

📊 Function that takes an array as an argument along with another argument and returns the average of the numbers passed through the array.

```
double getAverage(int arr[], int size) {
    int i;
    double avg;
    double sum;

    for(i = 0; i < size; ++i) {
```

```
        sum += arr[i];
    }

    avg = sum / size;
    return avg;
}
```

```
#include
/* function declaration */
double getAverage(int arr[], int size);

int main() {
   /* an int array with 5 elements */
   int balance[5] = {1000, 2, 3, 17, 50};
   double avg;

   /* pass pointer to the array as an argument */
   avg = getAverage(balance, 5);

   /* output the returned value */
   printf("Average value is: %f", avg);
   return 0;
}
```

Output:

```
Average value is: 214.400000
```

Processing Arrays

For certain applications, **assignment of initial values to elements** of an array is required.

- **Global Arrays**
  The array be defined globally (extern) to ensure storage is allocated throughout program execution.

- **Static Arrays**
  The array be defined locally as static to ensure values persist between function calls.

🎓 Student Marks Example

🎓 Program to display the average marks of each student, given the marks in 2 subjects for 3 students.

```
#include
#define SIZE 3
main() {
   int i = 0;
   float stud_marks1[SIZE]; /* subject 1 array declaration */
   float stud_marks2[SIZE]; /* subject 2 array declaration */
   float total_marks[SIZE];
   float avg[SIZE];

   printf("\nEnter marks in subject-1 out of 50 marks:\n");
   for(i = 0; i < SIZE; i++) {
      printf("Student no. =%d", i+1);
      printf("Please enter marks:");
      scanf("%f", &stud_marks2[i]);
   }

   for(i = 0; i < SIZE; i++) {
      total_marks[i] = stud_marks1[i] + stud_marks2[i];
      avg[i] = total_marks[i] / 2;
      printf("Student no.=%d, Average=%f\n", i+1, avg[i]);
   }
}
```

Output:

```
Enter marks in subject-1 out of 50 marks:
Student no. = 1 Enter marks= 23
Student no. = 2 Enter marks= 35
Student no. = 3 Enter marks= 42

Enter marks in subject-2 out of 50 marks:
Student no. = 1 Enter marks= 31
Student no. = 2 Enter marks= 35
Student no. = 3 Enter marks= 40

Student no. = 1 Average= 27.000000
Student no. = 2 Average= 35.000000
Student no. = 3 Average= 41.000000
```

🎯 Summary

Subscripts

Access individual elements using index from 0 to SIZE-1

Functions

Pass arrays using pointer, sized, or unsized parameters

Processing

Use global or static arrays for persistent data

Arrays provide a powerful mechanism for managing collections of related data efficiently in C programming!